

TelemetriaMqtt v2 — Checkpoint 3

Ambiente Docker local e broker MQTT (Mosquitto) com autenticação e controle de acesso

Projeto de Telemetria FSAE

16 de setembro de 2026

Abstract

Este documento detalha o terceiro checkpoint do projeto de reconstrução do sistema de telemetria da equipe: a criação de um ambiente de desenvolvimento local baseado em Docker e a configuração de um broker MQTT (Eclipse Mosquitto) com autenticação por usuário e controle de acesso por tópico (ACL). O checkpoint inclui também uma suíte de testes de integração automatizados que validam o comportamento real do broker, e uma correção em um componente do checkpoint anterior, descoberta durante a validação manual ponta a ponta.

Contents

1	Contexto e objetivo	1
2	Por que um broker MQTT com autenticação	2
3	Estrutura de arquivos criada	2
3.1	docker-compose.yml	3
3.2	mosquitto.conf	3
3.3	Controle de acesso por tópico (ACL)	4
3.4	Geração de credenciais	4
4	Testes de integração automatizados	5
4.1	Arquitetura da suíte de testes	5
4.2	Casos de teste e o que cada um comprova	5
4.3	Descoberta durante a execução: ACL de leitura não é aplicada na assinatura	5
4.4	Resultado da execução	6
5	Validação manual ponta a ponta	6
5.1	Bug encontrado e corrigido durante a validação manual	7
6	Resumo do que ficou pronto	7
7	Próximos passos	7

1 Contexto e objetivo

O projeto está sendo reconstruído do zero a partir do TCC anterior da equipe, seguindo uma metodologia de checkpoints documentada no arquivo `CLAUDE.md` do repositório: cada etapa é

implementada com testes antes ou junto do código (TDD), validada rodando de verdade (não só lendo o código), e só avança para a próxima etapa após revisão explícita.

Uma decisão de arquitetura anterior (documentada no `SCOPE.md`, tomada através de uma sessão de perguntas e respostas dedicada) definiu que **todos os serviços de backend** do projeto — broker MQTT, banco de dados de série temporal (InfluxDB), painel de visualização (Grafana) e o script de ingestão — seriam executados como **containers Docker**, tanto em desenvolvimento local quanto na VPS de produção. As razões principais foram:

- **Reprodutibilidade:** o mesmo ambiente roda idêntico no computador do desenvolvedor e no servidor, eliminando divergências de configuração.
- **Testabilidade:** cada checkpoint de backend pode ser validado subindo os containers localmente e testando o comportamento real, sem precisar de acesso à VPS a cada iteração.
- **Compatibilidade:** todas as imagens oficiais utilizadas (`eclipse-mosquitto`, `influxdb`, `grafana/grafana`) possuem suporte confirmado para arquitetura ARM64, compatível com a VPS gratuita escolhida (Oracle Cloud Free Tier).

O checkpoint 3 é a primeira aplicação prática dessa decisão: colocar o broker MQTT no ar, localmente, com autenticação e controle de acesso adequados — sem ainda envolver TLS, que fica reservado para quando o ambiente de produção (VPS) entrar em cena, através de um reverse proxy (Traefik).

2 Por que um broker MQTT com autenticação

Um broker MQTT é o intermediário entre quem publica dados (no caso final, o firmware do ESP32 instalado no carro) e quem consome esses dados (o painel ao vivo no Grafana e o script que grava o histórico no InfluxDB). Sem autenticação, qualquer dispositivo na mesma rede poderia publicar dados falsos de telemetria ou ler os dados reais do carro.

O projeto anterior da equipe (TCC original) cometeu um erro concreto nesse sentido: as credenciais do broker MQTT (um serviço gerenciado, HiveMQ Cloud) ficaram hardcoded em texto plano no firmware, e o repositório com esse código foi publicado publicamente no GitHub — expondo essas credenciais. Esse incidente foi identificado durante o levantamento de requisitos deste novo projeto e motivou duas decisões:

1. Nenhuma credencial é armazenada em arquivo versionado pelo Git, em nenhuma camada do projeto (firmware ou infraestrutura).
2. O broker passa a ser auto-hospedado (self-hosted), dentro do controle da própria equipe, em vez de depender de um serviço de terceiros.

3 Estrutura de arquivos criada

O checkpoint adicionou o diretório `infra/` ao repositório, com a seguinte estrutura:

Listing 1: Estrutura do diretório `infra/`

```
1  infra/
2    docker-compose.yml
3    mosquitto/
4      generate_passwd.sh
5    config/
6      mosquitto.conf
7      acl.conf
8      passwd                (gerado localmente, NUNCA versionado)
9  tests/
```

```
10     conftest.py
11     test_mosquitto_acl.py
12     requirements-test.txt
13     pytest.ini
```

3.1 docker-compose.yml

Define o serviço do broker como um container Docker, mapeando a porta padrão do MQTT (1883) e montando os arquivos de configuração como volume somente-leitura:

Listing 2: infra/docker-compose.yml

```
1  services:
2    mosquitto:
3      image: eclipse-mosquitto:2
4      container_name: telemetria-mosquitto
5      ports:
6        - "1883:1883"
7      volumes:
8        - ./mosquitto/config:/mosquitto/config:ro
9        - mosquitto-data:/mosquitto/data
10     restart: unless-stopped
11
12  volumes:
13    mosquitto-data:
```

Note-se que não há configuração de TLS aqui — por decisão de arquitetura, o ambiente de desenvolvimento local roda sem criptografia de transporte, já que a rede Docker do computador não é exposta à internet. A criptografia real (via Let's Encrypt, gerenciada por um reverse proxy Traefik) só entra em cena no ambiente de produção da VPS, em um checkpoint futuro.

3.2 mosquitto.conf

Listing 3: infra/mosquitto/config/mosquitto.conf

```
1  listener 1883
2  allow_anonymous false
3  password_file /mosquitto/config/passwd
4  acl_file /mosquitto/config/acl.conf
5
6  persistence true
7  persistence_location /mosquitto/data/
8  log_dest stdout
```

Os pontos relevantes desta configuração:

- `allow_anonymous false` — desativa qualquer conexão sem usuário/senha. Esse é o oposto do comportamento padrão do Mosquitto, que por conveniência de testes permite conexões anônimas.
- `password_file` — aponta para o arquivo de senhas com hash (nunca em texto plano), gerado por uma ferramenta própria do Mosquitto, explicada na Seção 3.4.
- `acl_file` — aponta para o arquivo de controle de acesso por tópico, explicado na Seção 3.3.
- `log_dest stdout` — os logs vão para a saída padrão do container, visíveis via `docker compose logs`, sem precisar de volume adicional para arquivo de log.

3.3 Controle de acesso por tópico (ACL)

O arquivo `acl.conf` define exatamente o que cada usuário autenticado pode fazer:

Listing 4: `infra/mosquitto/config/acl.conf`

```
1 # Dispositivos de telemetria (ESP32 real ou o mock publisher): so podem
2 # publicar no proprio topico, nunca ler -- nem esse nem qualquer outro.
3 user esp32-telemetria
4 topic write telemetria/esp32/data
5
6 # Consumidores (script de ingestao, Grafana Live via plugin MQTT): so
7 # podem ler o topico de telemetria, nunca publicar.
8 user telemetria_reader
9 topic read telemetria/esp32/data
```

Foram definidos dois papéis (usuários), seguindo o princípio de menor privilégio — cada identidade só tem exatamente a permissão de que precisa para sua função, nada além disso:

Usuário	Permissão	Quem usa
esp32-telemetria	write-only	Firmware do ESP32 (futuro) e o mock publisher
telemetria_reader	read-only	Script de ingestão e Grafana Live

Table 1: Usuários MQTT definidos e seus papéis

Nenhum dos dois usuários tem acesso administrativo ao broker. Qualquer combinação usuário/tópico que não apareça explicitamente no arquivo é **negada por padrão** — comportamento nativo do Mosquitto quando um arquivo de ACL está configurado.

3.4 Geração de credenciais

As senhas reais nunca são escritas em nenhum arquivo versionado. Em vez disso, o script `generate_passwd.sh` usa a própria ferramenta oficial do Mosquitto (`mosquitto_passwd`), executada dentro de um container Docker efêmero, para gerar o arquivo de senhas com hash:

Listing 5: `infra/mosquitto/generate_passwd.sh` (resumido)

```
1 docker run --rm \
2   -v "$CONFIG_DIR:/mosquitto/config" \
3   eclipse-mosquitto:2 \
4   mosquitto_passwd -b /mosquitto/config/passwd "$USERNAME" "$PASSWORD"
```

Isso evita duas coisas: (1) precisar instalar o Mosquitto localmente só para gerar um hash de senha, e (2) qualquer senha em texto plano tocar o disco fora do prompt interativo do terminal. O arquivo resultante, `infra/mosquitto/config/passwd`, está listado no `.gitignore` do projeto.

Detalhe prático encontrado durante a execução

Ao gerar o arquivo de senhas pela primeira vez, o `mosquitto_passwd` emitiu avisos sobre permissões do arquivo (recomendando `chmod 0700` e propriedade do usuário `root`). Seguir essa recomendação literalmente (`chmod 0600`) quebrou o broker: como o container roda como um usuário interno chamado `mosquitto`, diferente do usuário do sistema operacional host que gerou o arquivo, a permissão restritiva impediu o processo dentro do container de sequer abrir o arquivo (erro `"Unable to open pwfile"` nos logs). A permissão foi ajustada para `0644` (leitura para todos, escrita só para o dono), que funciona corretamente no cenário de bind-mount entre host e container com usuários diferentes. Esse é um exemplo concreto de por que os testes e a validação manual (Seção 5) são indispensáveis: o comportamento só ficou claro rodando o container de verdade, não apenas lendo a documentação.

4 Testes de integração automatizados

Diferente dos testes do checkpoint anterior (que validavam lógica pura em Python, sem depender de infraestrutura externa), este checkpoint exige testar o comportamento real de um serviço externo — o broker MQTT. A abordagem escolhida foi escrever testes de integração que sobem um Mosquitto **efêmero e real** (não simulado) para cada execução da suíte de testes, usando exatamente os arquivos de configuração de produção do projeto (`mosquitto.conf` e `acl.conf`), mas com um arquivo de senhas descartável, gerado apenas para o teste.

4.1 Arquitetura da suíte de testes

O arquivo `conftest.py` define uma *fixture* do pytest que:

1. Copia a configuração real do projeto para um diretório temporário.
2. Gera um arquivo de senhas de teste com os mesmos nomes de usuário reais (`esp32_telemetria`, `telemetria_reader`), mas senhas descartáveis.
3. Escolhe uma porta TCP livre no sistema (evitando conflito com um broker de desenvolvimento que porventura já esteja rodando).
4. Sobe um container Docker efêmero do Mosquitto, apontando para essa configuração.
5. Aguarda ativamente até a porta responder, antes de liberar os testes.
6. Ao final da suíte, remove o container e o diretório temporário — nenhum resíduo fica no sistema.

4.2 Casos de teste e o que cada um comprova

Foram escritos quatro testes, usando o protocolo MQTT versão 5 (que fornece códigos de motivo, ou *reason codes*, explícitos nas confirmações do broker — diferente da versão 3.1.1, que falha silenciosamente em vários casos de recusa):

- Teste 1** `test_esp32_user_can_publish_to_its_topic` — conecta como `esp32_telemetria` e publica no tópico permitido. Verifica que o código de motivo da confirmação de publicação (PUBACK) indica sucesso.
- Teste 2** `test_esp32_user_subscribes_but_never_receives_without_read_access` — conecta como `esp32_telemetria` (que só tem permissão de escrita) e tenta assinar o próprio tópico. Verifica que, mesmo que a assinatura seja aceita pelo broker, nenhuma mensagem chega de fato quando outro cliente publica no mesmo tópico.
- Teste 3** `test_reader_can_subscribe_and_receive` — conecta como `telemetria_reader`, assina o tópico, e confirma que recebe corretamente uma mensagem publicada por `esp32_telemetria`.
- Teste 4** `test_reader_cannot_publish` — conecta como `telemetria_reader` (que só tem permissão de leitura) e tenta publicar. Verifica que o código de motivo da confirmação indica recusa.

4.3 Descoberta durante a execução: ACL de leitura não é aplicada na assinatura

O Teste 2 originalmente foi escrito com uma expectativa diferente da que o código final reflete: a suposição inicial era de que o broker recusaria explicitamente a *assinatura* (`subscribe`) de um usuário sem permissão de leitura, retornando um código de erro na confirmação de assinatura

(SUBACK). Ao rodar o teste pela primeira vez, o resultado real foi diferente: o Mosquitto **aceita** a assinatura normalmente (retorna sucesso no SUBACK), e só aplica a restrição de ACL no momento de **entregar** cada mensagem que chega depois.

Isso significa que, do ponto de vista de quem programa um cliente MQTT, não existe um sinal de erro imediato ao tentar assinar um tópico sem permissão — a única forma de perceber a restrição é observar que nenhuma mensagem chega, mesmo com outros clientes publicando ativamente. O teste foi reescrito para validar exatamente esse comportamento (nome atual: `..._subscribes_but_never_receives_without_read_access`), e essa descoberta foi documentada no `SCOPE.md` do projeto para que não seja esquecida ou mal-interpretada como um bug em etapas futuras.

Uma segunda nuance encontrada durante a execução do Teste 1: o código de motivo retornado pelo broker para uma publicação bem-sucedida nem sempre é exatamente 0 (Success). Quando não há nenhum assinante conectado no momento da publicação, o Mosquitto retorna o código 16 ("*No matching subscribers*"), que ainda representa sucesso do ponto de vista da autorização — a publicação foi aceita, só não havia ninguém ouvindo. A regra correta, e a que os testes finais aplicam, é: códigos de motivo menores que 128 indicam sucesso (com ou sem entrega); códigos maiores ou iguais a 128 indicam recusa por falta de autorização.

4.4 Resultado da execução

Listing 6: Saída real de `pytest -v`

```
1 tests/test_mosquitto_acl.py::test_esp32_user_can_publish_to_its_topic PASSED
2 tests/test_mosquitto_acl.py::
    test_esp32_user_subscribes_but_never_receives_without_read_access PASSED
3 tests/test_mosquitto_acl.py::test_reader_can_subscribe_and_receive PASSED
4 tests/test_mosquitto_acl.py::test_reader_cannot_publish PASSED
5
6 4 passed in 8.66s
```

Após a execução, foi confirmado manualmente que nenhum container ou volume residual ficou no sistema (`docker ps -a` não lista nenhum container de teste).

5 Validação manual ponta a ponta

Além dos testes automatizados, o comportamento completo do sistema foi validado manualmente, subindo o ambiente de desenvolvimento real (`docker compose up`) e executando o script mock de publicação (desenvolvido no checkpoint anterior) contra o broker real, com um assinante (`mosquitto_sub`, executado dentro do próprio container) ouvindo o tópico:

Listing 7: Comando de validação manual

```
1 docker exec telemetria-mosquitto mosquitto_sub -h localhost -p 1883 \
2   -u telemetria_reader -P dev-reader-pw -t telemetria/esp32/data -C 3 &
3
4 python -m mock_publisher --host 127.0.0.1 --port 1883 --no-tls \
5   --username esp32-telemetry --password dev-publisher-pw --duration 2
```

O assinante recebeu corretamente três mensagens JSON publicadas pelo script mock, confirmando que autenticação, autorização e o fluxo de dados completo funcionam de ponta a ponta.

5.1 Bug encontrado e corrigido durante a validação manual

A primeira tentativa dessa validação falhou com um erro de SSL (*"EOF occurred in violation of protocol"*). A causa: o script `mock_publisher`, escrito no checkpoint anterior — antes da decisão de que o ambiente local rodaria sem TLS — sempre ativava TLS incondicionalmente ao conectar em modo não-simulado. Como o broker de desenvolvimento não fala TLS, a conexão falhava na camada de criptografia.

A correção foi adicionar uma opção de linha de comando `-no-tls` ao script, permitindo desativar TLS explicitamente ao apontar para o broker de desenvolvimento local, mantendo TLS como padrão (ativado) para uso contra o broker de produção na VPS:

Listing 8: Trecho da correção em `mock_publisher/__main__.py`

```
1 parser.add_argument(  
2     "--no-tls", action="store_true",  
3     help="desliga TLS (broker de dev local nao usa TLS; producao usa)",  
4 )  
5 ...  
6 if not args.no_tls:  
7     client.tls_set()
```

Esse ajuste ilustra o valor de validar manualmente além dos testes automatizados: a suíte de testes de integração do broker não exercitava o `mock_publisher` (ela usa clientes MQTT construídos diretamente nos próprios testes), então esse problema de integração entre os dois componentes só apareceu ao testar o fluxo completo manualmente.

6 Resumo do que ficou pronto

- Broker MQTT (Mosquitto) rodando como container Docker, com autenticação obrigatória e controle de acesso por tópico.
- Dois papéis de usuário bem definidos, seguindo o princípio de menor privilégio.
- Geração de credenciais sem nunca tocar em texto plano em disco versionado.
- Quatro testes de integração automatizados, validando o comportamento real do broker (não um comportamento assumido).
- Validação manual ponta a ponta confirmando o fluxo completo: publicação autenticada → broker → entrega autorizada.
- Uma correção de compatibilidade no componente do checkpoint anterior, descoberta e resolvida durante a validação.

7 Próximos passos

O próximo checkpoint do projeto (checkpoint 4, conforme o `CLAUDE.md`) é o script de ingestão, responsável por assinar o tópico de telemetria como o usuário `telemetria_reader` e gravar os dados recebidos no InfluxDB, iniciando o caminho de armazenamento histórico da arquitetura definida no `SCOPE.md`.